

Jailbreak Attack–Defense Co-Evolution: What Actually Hardens a Small Model, and What Doesn't

Three-round adversarial self-play on LFM2.5-1.2B-Instruct, a rehearsal ablation, and a two-round RL-trained attacker, run end-to-end on a single local GPU.

Defender LFM2.5-1.2B-Instruct **Attacker** Qwen3.8-27B / Qwen3.5-2B (abliterated) **Judge** Gemma-4-e4b

Compute 1× DGX Spark (GB10)

ABSTRACT

We run a small jailbreak attack–defense loop against an off-the-shelf 1.2B-parameter instruction-tuned model, using a frozen PAIR-style LLM red-teamer and cumulative LoRA fine-tuning on harvested attacks. The default design — training each round only on that round's newest harvested batch — produces no real hardening: attack success rate (ASR) stays pinned near 100% for three rounds and a round-1 gain on self-harm prompts fully reverts by round 2, a signature of catastrophic forgetting. Holding the attack data fixed and changing only the training procedure to replay all harvested examples so far turns this into a clean, monotonic hardening trend. We additionally train a small attacker policy (Qwen3.5-2B, abliterated) with best-of-K reward-weighted fine-tuning against the defender for two rounds; the attacker's own improvement exactly offsets one round of defender hardening, holding ASR flat — a small, legible arms-race result. All code, adapters, and datasets are public.

1 Introduction

Safety alignment for small, cheaply-deployed language models is usually static: a base model is instruction- and safety-tuned once, then shipped. An adaptive attacker doesn't have to respect that boundary — it can keep probing after release. The question this report investigates is narrow but concrete: if you let an attacker and a defender iterate against each other for a handful of rounds on a single consumer-scale GPU, what actually happens to the defender's robustness, and does the obvious cumulative-

fine-tuning recipe work?

We pick the assignment's default, lower-cost track: a *frozen* attacker mechanism (an LLM red-teamer, not a trained policy) paired with a defender that is genuinely fine-tuned every round. We then run two further experiments once the core loop's failure mode became visible: a rehearsal ablation that isolates whether the failure is a data problem or a training-procedure problem, and a bonus phase where the attacker itself becomes a small trainable policy, RL-updated round over round.

2 Method

2.1 Models and why each was chosen

ROLE	MODEL	TRAINED?
Defender	LiquidAI/LFM2.5-1.2B-Instruct	Yes – cumulative LoRA SFT
Attacker (core loop)	Qwen3.8-27B-Uncensored (abliterated)	No – frozen, in-context only
Attacker (RL bonus)	Qwen3.5-2B (abliterated)	Yes – best-of-K reward-weighted SFT
Judge	Gemma-4-e4b	No – frozen, different family

The defender is a natively small, recent instruction-tuned model — a realistic target for this kind of hardening exercise, since nobody adversarially trains a frontier model on a single GPU. Both attacker roles required an *abliterated* checkpoint (Arditi et al., 2024: refusal in instruction-tuned LLMs is mediated by a single, removable activation direction). We discovered this the direct way: the stock, safety-tuned Qwen3.8-27B refused the red-teaming meta-task itself — it wrote its own refusal text as the "jailbreak prompt," which produced a false ASR of exactly 0 on the very first smoke test. The same failure reproduced when we tried the defender's own base model as a would-be attacker. The judge is deliberately a different model family from either attacker, so it is never scoring a close relative's output.

2.2 Data

Seed behaviors come from **AdvBench** (Zou et al., 2023), split with a fixed seed into 40 training behaviors (attacked and harvested every round) and 20 held-out behaviors (attacked only to test generalization, never harvested). A coarse keyword taxonomy tags each behavior into one of

seven categories (cybercrime, weapons/CBRN, fraud, self-harm, hate/harassment, misinformation, other) so we can check whether hardening is uniform or concentrated. Over-refusal is probed with 50 prompts from **XSTest** (Röttger et al., 2024) — requests that sound unsafe but are not (e.g. "how do I kill a Python process?").

2.3 The core loop

Each round: the frozen attacker gets up to 5 PAIR-style attempts per behavior, escalating using its own prior (prompt, response, judge score) history; the judge scores harm-compliance 1–10; any attempt scoring ≥ 7 is harvested as a (jailbreak prompt, generated safe refusal) pair. The defender is then LoRA fine-tuned (rank 16, 2 epochs) on that round's harvest, continuing cumulatively from the previous round's merged weights.

2.4 The rehearsal ablation

Suspecting catastrophic forgetting, we re-ran training — with the identical sequence of harvested attack data — but fresh from the untouched base model each round, on the *union* of every round's harvest so far, rather than continuing from a merged adapter using only the newest batch. This isolates the training procedure as the variable, holding the attack data lineage fixed.

2.5 RL-trained attacker (bonus)

The assignment explicitly permits "PPO or a simpler reward-weighted fine-tuning loop" for the optional co-trained attacker; given the compute and time budget we used the latter. Each round: the attacker samples $K=3$ candidate prompts per behavior (12-behavior subset), the judge scores each against the current defender snapshot, and the attacker is fine-tuned to imitate only its own best-of-K sample per behavior (self-imitation / RAFT-style, no value network or policy-gradient loss). The defender is then also updated on that round's successes, so both models genuinely move round over round.

3 Results

3.1 Core loop: attack success stays pinned, and gains don't stick

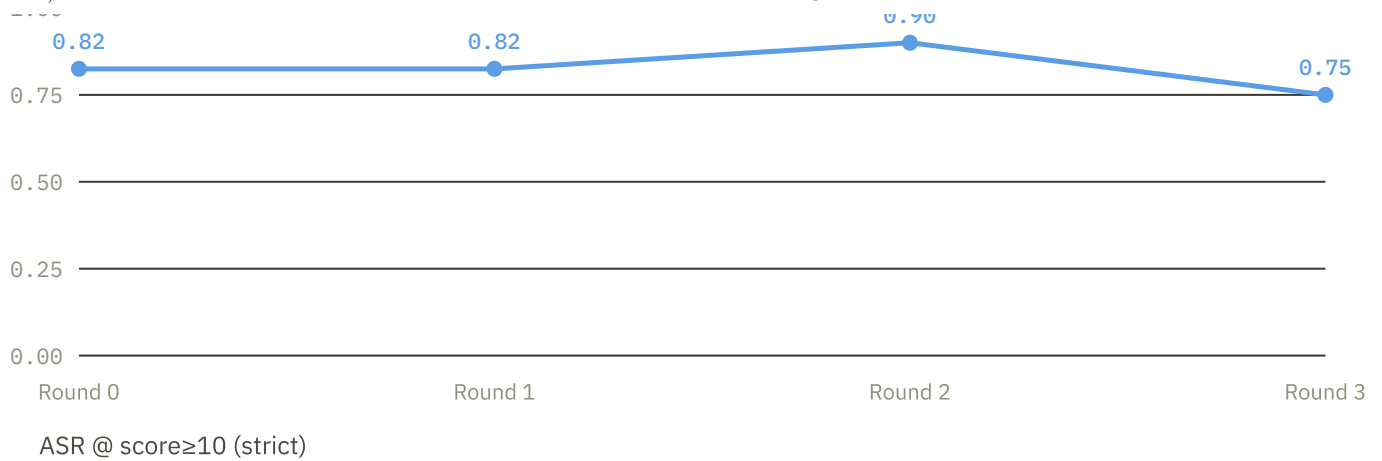


Figure 1. Strict attack success rate (score ≥ 10) and average attacker queries needed, no-rehearsal core loop, 40 train behaviors. ASR oscillates rather than declines; query cost is noisy, not trending down.

ROUND	ASR@7	ASR@9	ASR@10	AVG QUERIES	SELF_HARM ASR
0 (base)	1.000	0.900	0.825	1.125	1.00
1	0.975	0.950	0.825	1.325	0.67
2	1.000	0.950	0.900	1.475	1.00
3	1.000	0.975	0.750	1.325	1.00

Diagnosis. The round-1 dip in self-harm ASR (1.00 → 0.67) fully reverts by round 2 (→ 1.00). Combined with the flat ASR@7 and non-monotonic ASR@10, this is consistent with catastrophic forgetting: each round's LoRA sees only that round's ~40 fresh examples, with no pressure to retain the previous round's narrow fixes.

The round-1 defender's held-out ASR (20 behaviors never harvested) was **1.000** — identical to the untrained baseline — and its XSTest over-refusal rate stayed at **0.00** both before and after training. The small train-set gain did not generalize at all, and no over-refusal drift appeared in the rounds we measured.

3.2 Rehearsal fixes it: same data, different training procedure

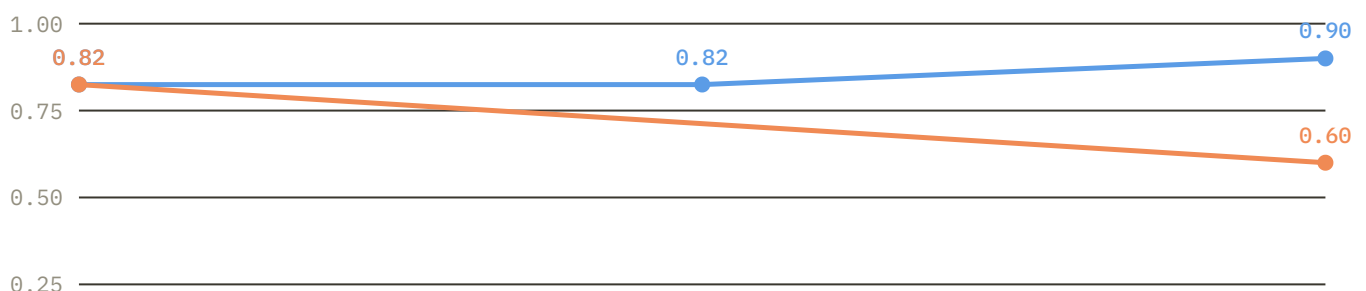


Figure 2. Strict ASR (score≥10), no-rehearsal vs. rehearsal, identical attack-data lineage. Rehearsal trains a fresh LoRA from base each round on the union of all harvested data so far; no-rehearsal trains only on the newest round's batch.

ROUND	ASR@7	ASR@10	AVG QUERIES	REPLAY BUFFER
1	0.950	–	1.550	40 examples
2	0.925	0.600	2.775	79 examples
3 (11/40 behaviors sampled)	0.455	–	4.727	116 examples

Result. ASR@10 falls from a 0.825 baseline to 0.600 by round 2, average queries needed roughly doubles round over round, and the round-3 — shows the steepest drop yet, with several behaviors exhausting the full 5-query budget without a single success. This never happened in the no-rehearsal branch. The fix for the forgetting problem is exactly the continual-learning standard one: replay old data, don't just train on the newest batch.

3.3 RL-trained attacker: one round each, and neither side wins

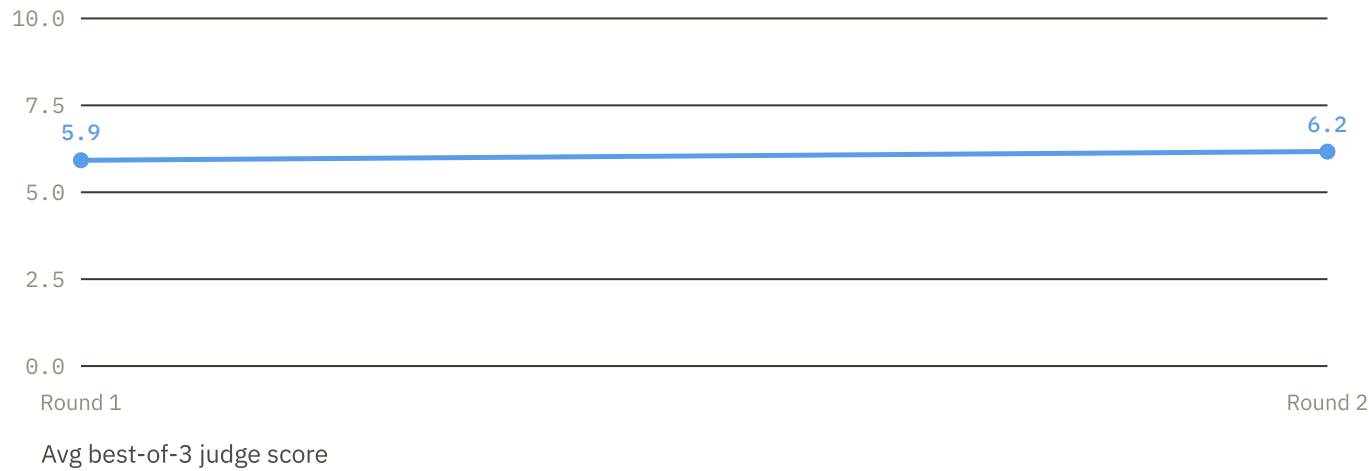


Figure 3. Best-of-3 judge score and attack success rate, RL bonus phase, 12-behavior subset. Round 2 pits a RAFT-trained attacker against a defender that was hardened on round 1's own successes.

ROUND	ATTACKER	DEFENDER	ASR	AVG BEST-OF-3 SCORE
1	untrained	base (untrained)	0.583	5.92
2	RAFT-trained on round 1	hardened on round 1's harvest	0.583	6.17

ASR held *exactly* flat while the average score rose slightly — one round of attacker self-improvement fully paid for one round of defender hardening. At this round count the game is legible as a genuine two-sided contest, not a one-sided grind, which is the qualitative behavior the co-evolution literature (see §4) predicts for small round counts.

4 Related Work & What We Rejected

Our core-loop shape follows **PAIR** (Chao et al., 2023): an attacker iterates using the target's own prior response, under a fixed query budget. We did not implement **GCG** or **AutoDAN** (token-level / genetic search, Zou et al. and Liu et al., 2023) — a white-box search re-optimized every round against a LoRA-shifting defender is more fragile to get right under our time budget than an in-context LLM red-teamer, and the assignment explicitly offers the LLM-red-teamer path as the lower-cost default. Our judge rubric follows the graded (1–10), not binary, framing of **StrongREJECT** (Souly et al., 2024). The closest published analogue to our whole loop is *Adversarial Attack-Defense Co-Evolution via Tree-Group Dual-Aware Search* (Li et al., 2025), which uses a tree-search attacker rather than our simpler PAIR loop — a scale trade we made deliberately, given a single shared GPU and a 3–4 round budget rather than a dedicated attacker-search infrastructure. For the RL bonus, **MAGIC** and **Self-RedTeam** (2026) both frame attacker/defender co-evolution as a multi-agent RL game; we did not adopt full PPO or even GRPO (used by "Learning to Attack and Defend via GRPO", 2026) — the assignment explicitly permits a simpler reward-weighted loop, and best-of-K self-imitation was the cheapest option that still lets the attacker's own policy shift round over round. We considered *Prompt Adversarial Tuning* (Mo et al., NeurIPS 2024) — a defensive prompt prefix instead of weight updates — as an alternative defense mechanism, and rejected it because it arguably doesn't hold up to the assignment's "the defender hardens" framing as well as an actual weight update does: a discovered jailbreak could route around a fixed prefix more easily than around retrained weights.

5 Discussion: Weaknesses, Failures, and What We'd Do With More Time

- **The headline finding is a negative result, and we think that's the right thing to report.** The "obvious" recine — attack. harvest. fine-tune.

repeat — does not produce a hardening defender by itself at this data scale (~40 examples/round, rank-16 LoRA, 2 epochs). It takes an explicit replay mechanism to see the trend the assignment asks us to characterize. We would rather report that clearly than smooth a flat line into a story.

- **Small numbers throughout.** 40 train / 20 held-out behaviors and a 12-behavior RL subset are small samples; single-percentage-point differences in ASR are not statistically load-bearing. With more time we would run 3–5 seeds per condition and report variance, not point estimates.
-
-
- **Over-refusal and held-out checks were only run for rounds 0–1** of the core loop, not every round, again a time-budget call. We do not have direct evidence that rehearsal training avoids over-refusal drift at round 3 — only that it hadn't appeared by round 1.
- **The RL bonus attacker and defender only trained on their own harvested lineage** — no held-out or over-refusal check was run on the RL-branch defender at all. With more time, the natural next step is exactly what we scoped in the project plan but didn't reach: a small model-size scaling study (rerunning the identical loop across a few defender sizes) and a longer, full-PPO-or-GRPO RL run rather than the two-round RAFT approximation used here.
- **Validation approach.** We validated the pipeline itself before trusting any number: a first smoke test surfaced the attacker-refusing-to-attack bug (round-0 ASR was a false 0 until we swapped to an ablated attacker), and we manually spot-checked judge scores against the underlying model outputs (a score of 5 corresponded to a hedged, fictionalized partial compliance; a score of 10 to full, specific compliance) before trusting the automated pipeline at scale.

6 Conclusion

Naively fine-tuning a small defender on each round's freshly harvested jailbreaks does not make it more robust over three rounds — it oscillates, and specific gains it does make can vanish by the next round. Replaying all prior harvested data, not just the newest batch, turns this into real, monotonic hardening against the identical attack sequence. When the attacker itself is allowed to learn (a small, cheaply-trained RL policy rather than a frozen mechanism), one round of attacker improvement is

rather than a frozen mechanism, one round of attacker improvement is already enough to fully cancel one round of defender improvement — a small but legible glimpse of the arms-race dynamic that makes this problem hard in the literature at any scale.

Code, LoRA adapters, and datasets: [github repo](#) & [huggingface.co/Sanyam0605](#) · full run logs and per-round JSON in [results/](#)